

Joint Task Offloading, Resource Allocation, and Service Caching in Mobile Edge Computing via Soft Actor-Critic Learning

Haizhou Bao¹, Yuxuan Wang, Yiming Huo², *Senior Member, IEEE*, Peng Li³, *Member, IEEE*,
Lei Nie⁴, *Member, IEEE*, Libing Wu⁵, *Senior Member, IEEE*, and Guorong Yu⁶

Abstract—Mobile edge computing (MEC) has emerged as a powerful solution for handling computation-intensive applications at the network edge. Task offloading has become a widely adopted strategy to optimize network resource utilization and deliver enhanced services to user equipment (UE). To meet stringent end-to-end delay and energy consumption requirements, users can offload tasks to edge servers where relevant services are pre-cached. However, the limited storage capacity and the heterogeneity of edge nodes make collaborative cache decision-making and computing resource allocation at the edge critical challenges. We address the optimization of task offloading, resource allocation, service caching, and push decisions within the MEC system through the coordinated efforts of edge servers. The objective is to minimize long-term offloading latency while maximizing the cache hit ratio, presenting a mixed-integer non-linear programming (MINLP) problem. To tackle this challenge, we propose a novel soft actor-critic approach, transforming the problem into a Markov decision process (MDP). Numerical results demonstrate that our algorithm significantly outperforms existing methods in terms of system cost, average delay, and cache hit ratio.

Index Terms—Mobile edge computing, computation offloading, soft actor-critic, MDP.

I. INTRODUCTION

IN RECENT years, the rapid advancement and widespread adoption of 5G networks and mobile smart devices have

brought various cutting-edge technologies, such as autonomous driving, virtual reality (VR), and augmented reality (AR), into real-world applications [1], [2], [3]. However, these technologies, particularly those with heavy computational demands, require substantial communication and processing resources at the network edge. To ensure user safety in critical applications like autonomous driving, these technologies must execute computational tasks with extremely low latency [4]. Minimizing latency and energy costs while maintaining a high-quality user experience is a significant challenge under these conditions.

Traditional cloud computing [5] has long been a well-established centralized computing model in practical use. The Internet of Things (IoT) benefits from advanced computation and caching capabilities made possible by the vast resources available on cloud servers. However, the physical distance between cloud servers and end-users introduces substantial transmission delays, which are inadequate for emerging applications with ultra-low latency requirements. Moreover, 5G/6G networks have the potential to deliver sophisticated artificial intelligence (AI) services to UEs, thereby enhancing system intelligence. Yet, UEs with limited energy and computing capacity struggle to fulfill the stringent demands of certain applications, such as autonomous driving.

MEC has emerged as a complement to cloud computing by bringing computation and caching resources closer to the network edge [6]. Applications running on UEs can transmit tasks to edge servers that offer greater computational power and superior service caching, thus reducing transmission costs and lowering task latency. Furthermore, unlike cloud computing, MEC allows users to perform computation tasks with enhanced security and reliability, since there is no need to transmit raw data over long distances [7]. Despite these advantages, MEC faces challenges due to the diversity of computational workloads and service requests, making task processing complex [8], [9]. Service caching in MEC is crucial, especially when UEs are constrained by limited energy and are highly sensitive to processing delays. Tasks generated by mobile edge nodes often require associated services at the edge nodes to provide service, including program codes, databases necessary for computation, and so on.

A large number of services related to tasks, such as object tracking, must be pre-cached on the edge server before task

Received 22 January 2025; revised 18 June 2025; accepted 17 August 2025. Date of publication 15 September 2025; date of current version 6 March 2026. This work was supported in part by the Natural Science Foundation of Hubei Province under Grant 2025AFB544 and in part by the Fundamental Research Funds for the Central Universities, Jilin University under Grant 93K172024K27. The review of this article was coordinated by Prof. Kyunghan Lee. (*Corresponding authors: Peng Li; Lei Nie.*)

Haizhou Bao is with the School of Artificial Intelligence and Computer Science, Hubei Normal University, Huangshi 435000, China, also with the School of Computer Science and Technology, Wuhan University of Science and Technology, Wuhan 430065, China, and also with the Key Laboratory of Symbolic Computation and Knowledge Engineering of Ministry of Education, Jilin University, Changchun 130012, China (e-mail: baohaizhou@wust.edu.cn).

Yuxuan Wang, Peng Li, Lei Nie, and Guorong Yu are with the School of Computer Science and Technology, Wuhan University of Science and Technology, Wuhan 430065, China (e-mail: wangyx@wust.edu.cn; lipeng@wust.edu.cn; lnjie@wust.edu.cn; yugorong190@wust.edu.cn).

Yiming Huo is with the Department of Electrical and Computer Engineering, University of Victoria, Victoria, BC V8W 2Y2, Canada (e-mail: ymh@uvic.ca).

Libing Wu is with the School of Cyber Science and Engineering, Wuhan University, Wuhan 430072, China (e-mail: wu@whu.edu.cn).

Digital Object Identifier 10.1109/TVT.2025.3610283

execution within the MEC system. However, the limited storage capacity of edge nodes necessitates dynamic decisions on which services to cache and where to execute tasks—locally or offloaded to the edge servers. Given the delay-sensitive nature of application tasks and the substantial transmission delay to cloud servers, we focus on optimizing task offloading scheduling at the network edge. The heterogeneity in CPU and storage capacities among edge nodes further complicates the decision-making process for computation and caching.

Integrated computation and service caching in MEC have emerged as promising solutions for resource-constrained edge computing. Specifically, the optimization of computation, caching, and communication (3 C) has garnered significant attention, focusing on service deployment, communication resource allocation, and offloading decisions. This challenge typically presents itself as a MINLP problem, which is generally an NP-hard problem. Some researchers have explored task offloading schemes combined with service placement in MEC [10], [11]. However, many of these approaches consider multiple types of services and often rely on static caching schemes without incorporating cache pushing, rendering them less practical for dynamic MEC environments. Although some studies have examined service caching and pushing in conjunction with edge computation [12], [13], they tend to overlook the collaboration between edge nodes for service caching and computing. Additionally, traditional optimization methods face inefficiencies in resolving joint optimization problems in dynamic MEC environments. Consequently, numerous researchers have adopted deep reinforcement learning (DRL) to address dynamic caching and offloading in MEC systems [14], [15]. The computational complexity of joint caching deployment differentiates it from other DRL-based offloading challenges in MEC. Moreover, reinforcement learning approaches often face the curse of dimensionality, and classic DRL methods struggle to manage high-dimensional optimization problems effectively [12], [16].

In this paper, we study cooperative MEC (Mobile Edge Computing) caching with dynamic task offloading by integrating caching history and environmental mobility into a unified decision-making framework. We address the joint optimization of task offloading ratios, resource allocation, caching placement, and update strategies within the MEC system. Our objective is to minimize the overall execution delay while maximizing the cache hit ratio, subject to the constraints of edge nodes' caching and computing capabilities, caching decisions, and caching update policies. To achieve this, we propose an enhanced Soft Actor-Critic (SAC) method that improves service-empowered offloading and boosts task offloading efficiency. The key contributions of this paper are organized as follows:

- We propose a multi-edge server, multi-user collaborative MEC system model, where tasks can be offloaded to multiple edge servers for collaborative processing. The joint optimization of computation offloading, service caching, and resource allocation is formulated as an MINLP problem aimed at minimizing the total system cost.
- We introduce an improved soft actor-critic (ISAC)-based approach to address the optimization challenge by formulating it as a MDP. The approach incorporates an entropy

term into the objective function to enhance exploration during training and avoid convergence to suboptimal solutions. The actor module adopts a multi-branch architecture, where separate networks generate actions for different users, and integrates long short-term memory (LSTM) to enable the agent to learn long-term content popularity patterns in dynamic environments, thereby providing improved strategies for task offloading, service caching, and resource allocation.

- We verify the efficacy of the proposed ISAC method using simulated tests. The findings indicate that our approach outperforms existing benchmark algorithms in terms of long-term cost, average latency, and cache hit ratio under various circumstances.

The rest of this paper is arranged as follows. Section II provides an overview of related work. In Section III, we present the problem formulation and solution. Section IV describes the proposed solution and algorithm. Section V evaluates the performance of the proposed algorithm. Finally, the conclusion is provided in Section VI.

II. RELATED WORK

The rapid growth of the IoT and advancements in communication technologies have significantly contributed to the development of edge computing. The large-scale, delay-sensitive tasks generated by edge nodes face various challenges, such as resource constraints and energy limitations, which have received considerable attention. In this section, we review existing research on deep reinforcement learning-based offloading and service-based offloading methods, focusing particularly on service-based offloading around edge nodes, excluding the involvement of cloud computing. We compare the existing research with our proposed problem to illustrate the novelty of this paper as shown in Table I.

A. Task Offloading in MEC

A substantial body of work has explored computing offloading using DRL-based methods [17], [18], [27], [28]. For instance, Zhang et al. [17] examined a cooperative partial task offloading and resource allocation scheme in a cloud-edge-end-based Industrial Internet of Things (IIoT), seeking to reduce long-term latency and energy costs. They suggested an upgraded decentralized multi-agent deep reinforcement learning (MADRL) technique to solve this challenge. Wang et al. [27] studied task offloading and resource management for a platoon network and suggested an optimization strategy combining task offloading and power allocation. Considering the complexity of offloading in MEC, Yang et al. [28] studied maximizing the satisfaction of users within the C-RAN architecture with multi-layer MEC, proposing two approximation techniques to solve the problem in two phases. Similarly, Yang et al. [18] explored a complex task offloading issue in MEC, considering both energy consumption and latency, and proposed an efficient multi-task evolutionary algorithm to address the challenge. While prior works often abstract away wireless link constraints, our framework explicitly models and jointly optimizes task offloading,

TABLE I
COMPARISON OF OFFLOADING SCHEME

Reference	Partial offloading	Resource Allocation	Service Caching	Multiple Users and Multiple BSs	Coordination Between BS	Service Pushing	Objective
Zhang <i>et al.</i> [17]	✓	✓	×	✓	✓	×	Weighted sum of the delay and energy cost
Yang <i>et al.</i> [18]	×	✓	×	✓	×	×	Processing time and energy consumption
Zhang <i>et al.</i> [19]	✓	×	✓	×	×	×	Average delay
Song <i>et al.</i> [20]	×	×	✓	×	×	×	Operator's utility
Gao <i>et al.</i> [21]	×	×	✓	✓	×	×	Utility function
Zhang <i>et al.</i> [22]	×	✓	×	×	×	×	Execution time and consumed energy
Feng <i>et al.</i> [23]	×	✓	×	×	✓	×	Weighted bandwidth, delay, and energy consumption
Fan <i>et al.</i> [24]	×	✓	✓	✓	✓	×	Total task processing latency
Zhao <i>et al.</i> [25]	✓	✓	✓	✓	×	×	Total computation delay
Zhou <i>et al.</i> [26]	✓	✓	✓	✓	✓	×	Energy consumption
Ours	✓	✓	✓	✓	✓	✓	Long term latency and cache hit ratio

caching policies, and wireless resource allocation to ensure robust, service-empowered edge computing even under stringent resource scarcity.

B. Joint Task Offloading and Service Caching

Given that service data is essential for task computation, task offloading schemes must consider service deployment [19], [20], [29], [30]. For example, Teng *et al.* [29] investigated the optimum task offloading and service placement challenge to maximize energy efficiency, presenting a break-point scheme and a convex programming-based algorithm to enhance energy efficiency. Zhang *et al.* [19] analyzed the performance of two offloading techniques for IAB-based MEC networks and conventional cellular MEC networks, determining the average latency for both strategies using stochastic geometry and queuing theory. Additionally, Song *et al.* [20] explored a recommendation of users and offloading strategies, assisted by device-to-device communication, aiming to optimize the operator's utility. Addressing execution mode, data transmission path, subchannel assignment, caching, and caching update strategies, Liu *et al.* [30] employed a simulated annealing genetic algorithm and a deep q network (DQN) to solve the task offloading problem across different time periods. To minimize costs and improve resource sharing efficiency in mobile cloud-edge computing, Zhou *et al.* [31] proposed a reverse auction-based offloading and resource allocation scheme that achieves near-optimal costs with reduced complexity. Service caching plays a vital role in reducing processing latency and enhancing task offloading efficiency. However, the lack of effective resource management and inter-node cooperation among edge servers is often overlooked in existing work, potentially leading to suboptimal system performance.

C. Joint Optimization of Service Caching, Task Offloading, and Resource Allocation

The joint optimization of service caching, task offloading, and resource allocation in the MEC scenario has been extensively studied. To mitigate the pressure on the network edges, Gao *et al.* [21] examined the utility maximization in terms of task offloading, content caching, and resource allocation, and then proposed a three-stage Stackelberg game to explore the interaction among cloud server, base station and mobile device. To address the different timelines of offloading and caching, Zhang *et al.* [22] explored a dual-timescale resource allocation problem combined with cooperative service caching and computation offloading, and proposed a hierarchical deep reinforcement learning method to solve the collaborative caching and task

offloading action. Feng *et al.* [23] analyzed the computation offloading, age of information (AoI) metric, and service caching joint optimization problem, subsequently proposing a service-assisted task offloading scheme. In a multitask and multiservice scenario, Fan *et al.* [24] studied a cooperative approach to service deployment, task scheduling, and resource allocation, to reduce total processing latency while maintaining task queuing stability over time. Nevertheless, as the number of optimization variables increases, the corresponding action space expands significantly. Existing studies often overlook this complexity when jointly optimizing service caching, task offloading, and resource allocation, which can hinder scalability and learning efficiency in dynamic environments.

D. Reinforcement Learning-Based Computation Offloading

Task offloading often faces challenges in time-varying environments with large state and action spaces, and typically lacks prior knowledge of the dynamic conditions. This has led to significant interest in the potential of DRL for task offloading, leading to numerous studies in this area [32], [33]. For instance, Zhou *et al.* [32] proposed a DRL-based task offloading and service placement scheme aimed at minimizing system costs while satisfying delay requirements. Similarly, the authors in [33] transformed the task offloading problem into an MDP and applied a DQN to reduce latency in MEC systems.

However, resource allocation variables are typically continuous, and using the DQN algorithm requires discretizing these continuous variables, which can lead to decreased solution accuracy and issues related to the curse of dimensionality. To address the challenge of partial offloading with continuous variables, extensive research has been conducted [25], [26]. For example, Zhao *et al.* [25] introduced a task offloading scheme with a cache-assisted framework, and developed a task pre-processing using a deep deterministic policy gradient algorithm. Zhou *et al.* [26] explored task offloading with service caching and resource allocation in a collaborative MEC system, employing a deep deterministic policy gradient (DDPG) approach to address the problem. Nevertheless, as the number of optimization variables increases, the corresponding action space expands significantly. Existing studies often overlook this complexity when jointly optimizing service caching, task offloading, and resource allocation, which can hinder scalability and learning efficiency in dynamic environments.

However, the DDPG method, which utilizes an actor-critic structure, is prone to overestimating the value function, which

can impair training effectiveness. Additionally, similar to other DRL algorithms, DDPG is susceptible to falling into local optima during the training process due to the large action space. To address these issues, the authors in [34] adopted a soft actor-critic (SAC) algorithm to manage task processing in vehicular networks. Unlike the DDPG algorithm, the SAC algorithm is a stochastic policy DRL approach that incorporates an entropy term in the objective function to prevent the training process from converging to local optima. Heidarpour et al. [16] focused on maximizing the lifetime of a MEC-enabled IoT system and proposed a SAC-based DRL approach for lifetime maximization. Considering task dependencies, Zhou et al. [35] investigated a task offloading scheme based on directed acyclic graphs (DAG) with soft cooperation in MEC systems, and proposed a branch soft actor-critic method to achieve optimal decision-making. Although deep reinforcement learning has been widely applied to task offloading problems, existing works frequently overlook the critical cooperation between multiple edge servers, which is essential for enhancing overall system performance.

III. SYSTEM MODEL

A. Network Model

In this paper, we examine a situation that includes several edge servers and various UEs. The system time is divided into multiple time slots while the time set is denoted as $\mathcal{T} = \{1, 2, \dots, T\}$. Edge servers have a large cache capacity and can store all of the service types necessary for computing activities, however, users can only store a limited number of services due to memory capacity limits. If a task without the related service cached locally is required to be calculated, it cannot be executed locally in the current time slot and must be retrieved from the edge server instead. Edge servers are more capable of processing computation tasks than UEs, and users can offload tasks to edge servers to minimize latency by leveraging edge servers' strong computing capabilities. Each edge server can collaborate and interact with others to execute computation tasks, ensuring a balanced use of computational resources. Each UE first transmits part of the task to nearby edge server m , if the task cannot be processed by the edge server, part of the task will be forwarded to other edge servers.

Fig. 1 shows the system schematic diagram. There are three BSs, each equipped with one edge server: $ES1$, $ES2$, and $ES3$. Here, we assume that all the base stations are connected by optical fiber. When the nearby ES cannot process the offloaded task, the task will be offloaded to other BSs. For example, when the task of $UE4$ cannot be processed locally, a portion of the task will be offloaded to the nearby edge server $ES1$ or $ES3$. Particularly, when the task cannot be processed on the edge server $ES1$, the task will be offloaded to the collaborative edge server $ES2$ for processing. The offloading flow is shown as the red dashed line in Fig. 1. On the other hand, some parameters for the offloading and service actions in the DRL can be trained in a distributed manner on each UE after interacting with the environment. In addition, the notation is depicted in Table II.

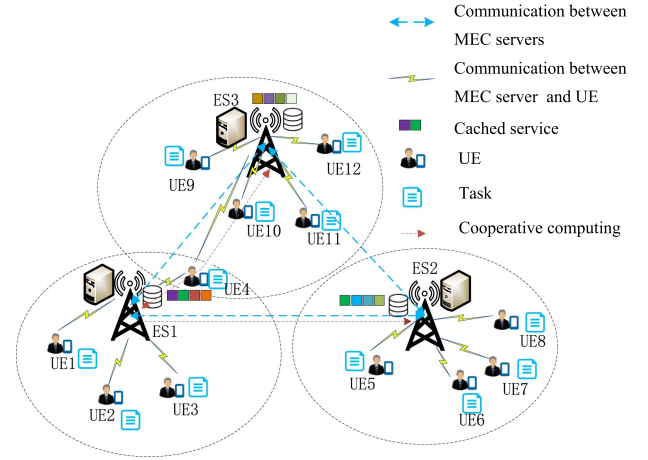


Fig. 1. Structure diagram of MEC system.

TABLE II
LIST OF NOTATIONS USED IN OUR WORK

Notation	Description
$\mathcal{T}$	Time slot set
$\mathcal{U}$	User set
$\mathcal{M}$	Edge node set
$l_{i,t}^t$	Data size for task Θ_i at time slot t
$c_{i,t}^t$	CPU computation capacity for task Θ_i at time t
$e_{i,t}^t$	Maximum energy constraint for task Θ_i at time t
$x_{i,m}^t, x_{i,k}^t$	Offloading decision for task Θ_i on edge node m and k at time slot t
$y_{i,m}^t$	Cache decision of task Θ_i on edge node m at time t
$y_{i,m}^{p,t}$	Pushing decision for task Θ_i at time slot t
$\Delta y_{i,m}^t$	Service cache update decision for task Θ_i on edge node m at time t
$H(t)$	Service cache hit ratio
μ_i	Data size of the service for task Θ_i
D_m	Maximum caching capacity for edge node m
$z_{i,m}^t$	Computation resource allocation decision of task Θ_i on edge node m at time slot t
$W_{i,m}^t$	Communication bandwidth between user i and edge server m at time slot t
$g_{i,m}^t$	Channel coefficient between user i and edge server m at time slot t
p_i	Transmission power of user i at time slot t
F_m^t	Maximum computing capacity of edge node m
$q_{i,m}^t$	Task set arrived at server m before the task Θ_i at time slot t
$q_{i,k}^t$	Task set arrived at cooperative edge server k before the task Θ_i at time slot t

The UEs generate a task at the beginning of each time slot t , and the task requires the corresponding services to compute the task. In addition, the task is latency-sensitive due to the critical importance of ensuring the Quality of Service (QoS) for some applications. For instance, healthcare monitoring applications, such as ICU monitoring, remote surgery, and wearable analytics, require real-time data analysis at the edge to reduce latency and ensure timely responsiveness during emergencies [36]. Additionally, real-time communication applications, such as smart city surveillance and AR-based IoT environments, benefit from dynamic service caching, as sensor data in these systems often exhibit spatial and temporal locality. This allows edge servers to cache and serve data dynamically, improving overall efficiency. Furthermore, intelligent transportation systems (ITS)

utilize dynamic service function chaining across MEC nodes for dynamic traffic management, vehicular service chaining, and connected vehicle applications, ensuring the seamless integration of real-time traffic monitoring, safety alerts, and route optimization [37]. Therefore, we only consider collaborative computation on the edge nodes; however, the system model can be extended to incorporate cloud computing for large-scale applications. Since the task corresponds to the UE at each time slot, we utilize $\mathcal{I} = \{i, \dots, I\}$ to represent the set of the UE and Θ_i to denote the set of task i , and utilize the set $\mathcal{M} = \{0, \dots, m, \dots, M\}$ to indicate the UE and the edge server sets, where 0 represents the UE i . Each task is represented as a quaternion $(l_i^t, c_i^t, e_i^t, y_{i,m}^t)$, which specifies the size of the task (in bits), the CPU cycles necessary to process each bit of data (in cycles/bit), the energy consumption after the task is fully computed (J), and the required service cache for task Θ_i at time t . The service size of the task Θ_i is μ_i . At the beginning of t , the controller confirms the caching situation of the edge nodes. Note that we use edge nodes to refer to the users and edge servers. Let $y_{i,m}^t = 1$ indicate that the service data of task Θ_i is cached by the edge node m (user or edge servers) at the time slot t , and 0 otherwise. Additionally, the maximum computation power of the edge node m is set to F_m^t .

In this paper, we jointly consider task offloading, service data caching and updating, and computation resource allocation to minimize the execution delay of all tasks in the long term and maximize the cache hit ratio. We use a set $\mathbf{X}(t)$ and $\mathbf{Z}(t)$ to denote task offloading and resource allocation decisions, respectively, and a set $\mathbf{Y}(t)$ to denote the service caching decision.

1) *Computation Offloading*: $\mathbf{X}(t) = \{x_{i,m}^t | i \in \mathcal{I}, m \in \mathcal{M}\}$, and $x_{i,m}^t \in [0, 1]$, where $x_{i,m}^t$ is the proportion of the task Θ_i uploaded to the edge server m . To ensure that each task can be processed fully at the network edge, it is necessary to satisfy the constraint as follows:

$$\sum_{m \in \mathcal{M}} x_{i,m}^t = 1, \forall t \in \mathcal{T}. \quad (1)$$

2) *Service Caching Decision*: $\mathbf{Y}(t) = \{y_{i,m}^t, \Delta y_{i,m}^t, y_{i,m}^{p,t} | i \in \mathcal{I}, m \in \mathcal{M}\}$. Service caching variables $y_{i,m}^t \in \{0, 1\}$, i.e., $y_{i,m}^t = 1$ represents that the service data of task Θ_i requested is stored on the edge node m , and 0 otherwise. Furthermore, we also consider the service cache update decision as described in [12], that is, $\Delta y_{i,m}^t \in \{-1, 0, 1\}$ is used to represent the cache update decision for the task Θ_i on the edge node m . Specifically, $\Delta y_{i,m}^t = -1$ indicates that the current edge nodes will delete the service of task Θ_i at the next time interval, and $\Delta y_{i,m}^t = 0$ represents that the cache will be retained at the next time interval. Finally, $\Delta y_{i,m}^t = 1$ denotes the service data of task Θ_i will be cached in the edge node m at the next time interval. On the other hand, we also consider proactive caching, where the edge node m preemptively caches certain services from the remote server, which stores all the services. We use $y_{i,m}^{p,t}$ to represent the proactive cache pushing decision. $y_{i,m}^{p,t} = 1$ indicates that the service data of task Θ_i will be pushed to the edge node m from the remote server at the end of time slot t , and $y_{i,m}^{p,t} = 0$ otherwise.

Since the maximum storage capacity of the edge node m is D_m , the cache size constraint is given by

$$\sum_{i \in \mathcal{I}} y_{i,m}^t \mu_i \leq D_m, \forall m \in \mathcal{M}, \forall t \in \mathcal{T}. \quad (2)$$

On the one hand, to ensure that the task Θ_i is successfully computed at the edge nodes, the service data of the task Θ_i should be cached on at least one edge node, that is

$$\sum_{m \in \mathcal{M}} y_{i,m}^t \geq 1, \forall i \in \mathcal{I}, \forall t \in \mathcal{T}. \quad (3)$$

According to the cache updating policy of the edge node, the cache capacity of edge node m after updating is given by

$$y_{i,m}^{t+1} = y_{i,m}^t + \Delta y_{i,m}^t. \quad (4)$$

On the other hand, the value of the cache update variable $\Delta y_{i,m}^t$ needs to satisfy the following constraints:

$$-y_{i,m}^t \leq \Delta y_{i,m}^t \leq \min(y_{i,m}^{p,t}, 1 - y_{i,m}^t), \quad (5)$$

where the left-hand side of (5) restricts that the service data of the task Θ_i can only be removed if it was cached previously, while the right side of (5) indicates that the service of the task is pushed to the UE from other edge servers when the service data has not been cached previously.

To maximize the offloading success rate at the network edge, the service data of the tasks generated by UEs should be cached at the network edge. Therefore, we take the service cache hit ratio as a metric to assess the performance of the cache deployment policy, which is given by

$$H(t) = \frac{\sum_{i \in \mathcal{I}} \sum_{m \in \mathcal{M}} y_{i,m}^t \mu_i}{\sum_{m \in \mathcal{M}} D_m}, \forall t \in \mathcal{T}. \quad (6)$$

3) *Computation Resource Decisions*: $\mathbf{Z}(t) = \{z_{i,m}^t | i \in \mathcal{I}, m \in \mathcal{M}\}$, and let $z_{i,m}^t \in (0, 1]$ represent the computation resources allocated to the edge node m at the time slot t , which is the proportion of the maximum computation resource used for task Θ_i .

B. Communication Model

To reduce mutual interference between the edge nodes, we assume that UEs can communicate with the edge server using the orthogonal frequency division multiple access (OFDMA) technique. According to Shannon's theorem, the uplink data transmission rate between the UE i and its directly connected edge server m is expressed as

$$R_{i,m}^t = W_{i,m}^t \log_2 \left(1 + \frac{p_i |g_{i,m}^t|^2}{\sigma^2} \right), \quad (7)$$

where $W_{i,m}^t$ and $|g_{i,m}^t|^2$ represent the communication bandwidth and channel gain between the UE i and its nearby edge server m ($m \in \mathcal{V}_i$), $\mathcal{V}_i$ is the set of edge servers that are directly connected to user i , and p_i denotes the transmission power of the UE, and σ^2 is the noise power.

When a user offloads a task to edge servers, the task is first uploaded to the connected edge server and then transmitted to

the target edge server. To simplify the model, the transmission speed between the edge servers is assumed to be R_{e2e}^t .

C. Task Computation Model

1) *Local Computation*: The computation latency required for a task computed on a UE $i(0)$ is denoted as

$$T_{i,0}^l(t) = \frac{x_{i,0}^t l_i^t c_i^t}{z_{i,0}^t F_0^t}, \quad (8)$$

where $x_{i,0}^t$ denotes the proportion of task Θ_i computed locally, l_i^t and c_i^t denote the data size and the CPU cycles for each bit of data, respectively. In addition, $z_{i,0}^t$ is the proportion of the computation resource allocated to the UE $i(0)$, which determines the actual computation rate of the UE, and F_0^t represents the maximum computation capacity of the UE i .

When the partial task is computed on the UE i , the energy consumption [9], [38] can be calculated by

$$\begin{aligned} E_{i,0}^l(t) &= \kappa x_{i,0}^t l_i^t c_i^t (z_{i,0}^t F_0^t)^3 T_{i,0}^l(t) \\ &= \kappa x_{i,0}^t l_i^t c_i^t (z_{i,0}^t F_0^t)^2, \end{aligned} \quad (9)$$

where κ denotes the energy efficiency factor.

Furthermore, the computation resource allocation for task processing on any edge node should satisfy the following constraint:

$$\sum_{i \in \mathcal{I}} z_{i,m}^t \leq 1, \forall m \in \mathcal{M}, t \in \mathcal{T}. \quad (10)$$

2) *Partial Offloading*: When tasks cannot be computed by the local user, the task will be offloaded to the nearby edge server m first. Since the computation result is usually very small, the return latency of the calculation results is ignored as in [39]. In summary, if a partial task is executed on the edge node m ($m \in \mathcal{V}_i$), the processing time includes: upload transmission delay $T_{i,m}^{trans}(t)$, the waiting time before computing the task Θ_m $T_{i,m}^{wait}(t)$, and the computation latency for task Θ_m $T_{i,m}^{comp}(t)$.

Firstly, part of the tasks is required to be delivered to the connected edge server. Thus, the transmission time from the UE i to the edge server m is given by

$$T_{i,m}^{trans}(t) = \frac{x_{i,m}^t l_i^t}{R_{i,m}^t}. \quad (11)$$

Considering that the edge server receives multiple requests for computation processing in a time slot, we assume that the tasks are handled in the order of arrival, and the waiting time before computing can be expressed as

$$T_{i,m}^{wait}(t) = \sum_{q \in \mathcal{L}_{i,m}(t)} \frac{x_{q,m}^t l_q^t c_q^t}{z_{i,m}^t F_m^t}, \quad (12)$$

where $\mathcal{L}_{i,m}(t)$ denotes the task queue set in which the task arrived at server m before the task Θ_i at the time slot t , furthermore, the wait time is the time duration of the computation time of all the tasks arrived before the current task Θ_i in the queue. $x_{q,m}^t$ indicates that portion of the task q offload the corresponding edge server m to process.

Consequently, the computation delay for the partially offload task Θ_i on the edge server m is given by

$$T_{i,m}^{comp}(t) = \frac{x_{i,m}^t l_i^t c_i^t}{z_{i,m}^t F_m^t}. \quad (13)$$

Accordingly, the overall latency to process the task Θ_i on the edge node m is given by

$$T_{i,m}(t) = T_{i,m}^{trans}(t) + T_{i,m}^{wait}(t) + T_{i,m}^{comp}(t). \quad (14)$$

The overall energy consumption to process task Θ_i on edge server m can be expressed by

$$\begin{aligned} E_{i,m}(t) &= E_{i,m}^{trans}(t) + E_{i,m}^{comp}(t) \\ &= p_i \frac{x_{i,m}^t l_i^t}{R_{i,m}^t} + p_m \frac{x_{i,m}^t l_i^t c_i^t}{z_{i,m}^t F_m^t}, \end{aligned} \quad (15)$$

where $x_{i,m}^t l_i^t$ is the offloading data size, and $x_{i,m}^t l_i^t c_i^t$ is CPU cycles for processing the offloaded task. $E_{i,m}^{comp}(t)$ is transmission energy consumption, and $E_{i,m}^{comp}(t)$ is computation energy consumption, p_m is the computation power of edge server m .

3) *Collaborative Computing*: If the task Θ_i cannot be processed on the edge server m , the partial task will be transmitted to another collaborative edge server k for processing, where $k \in \mathcal{M}'$ ($\mathcal{M}' = \{\mathcal{M} \setminus \mathcal{V}_i \setminus 0\}$). We use $x_{i,k}^t$ to denote the coordination indicator, $x_{i,k}^t = 1$ indicates that the task Θ_i will be offloaded to the cooperative edge server k , otherwise 0. The transmission time between edge server m and the collaborative edge server k is given by

$$T_{i,k}^{trans}(t) = \frac{x_{i,k}^t l_i^t}{R_{i,m}^t} + \frac{x_{i,k}^t l_i^t}{R_{e2e}^t}, k \in \mathcal{M}'. \quad (16)$$

Correspondingly, the time to wait for computation on the collaborative edge server computation

$$T_{i,k}^{wait}(t) = \sum_{q \in \mathcal{L}_{i,k}^t} \frac{x_{q,k}^t l_q^t c_q^t}{z_{i,k}^t F_k^t}, k \in \mathcal{M}', \quad (17)$$

where $\mathcal{L}_{i,k}^t$ denotes the task queue set, which consists of tasks that arrived at the edge server k before the current task Θ_i at the time slot t .

The computation latency on the collaborative edge server k is given by

$$T_{i,k}^{comp}(t) = \frac{x_{i,k}^t l_i^t c_i^t}{z_{i,k}^t F_k^t}, k \in \mathcal{M}'. \quad (18)$$

Therefore, overall latency to execute the task Θ_i on the collaborative edge server k can be expressed by

$$T_{i,k}(t) = T_{i,k}^{trans}(t) + T_{i,k}^{wait}(t) + T_{i,k}^{comp}(t), k \in \mathcal{M}'. \quad (19)$$

If the task Θ_i cannot be processed on the edge server m and locally, the task will be delivered to edge server k through the nearby edge server m , the energy consumption to process the task Θ_i on edge server k [26] is given by

$$E_{i,k}(t) = E_{i,m}^{trans}(t) + E_{m,k}^{trans}(t) + E_k^{comp}(t)$$

$$= p_i \frac{x_{i,k}^t l_i^t}{R_{i,m}^t} + p_m \frac{x_{i,k}^t l_i^t}{R_{e2e}^t} + p_k \frac{x_{i,k}^t l_i^t c_i^t}{z_{i,k} F_k^t}, k \in \mathcal{M}', \quad (20)$$

where $E_{i,m}^{trans}(t)$ is the transmission latency from UE i to the edge server m , $E_{m,k}^{trans}$ is the transmission latency between the edge server m and the collaborative edge server k , and $E_k^{comp}(t)$ is the computation delay of task Θ_i on the edge server k , and p_k is the computation power of the edge server.

In summary, the overall execution time of task Θ_i at the network edge includes three parts: the computation delay on the UE i , the processing delay on the edge server m , and the processing delay on the collaborative edge server k . Thus, the total execution delay for the task Θ_i is given by

$$T_i^{total}(t) = y_{i,0}^t T_{i,0}^l(t) + \sum_{m \in \mathcal{V}_i} y_{i,m}^t T_{i,m}(t) + \sum_{k \in \mathcal{M}'} y_{i,k}^t T_{i,k}(t). \quad (21)$$

The execution energy of the task Θ_i on the edge nodes also includes three parts, which are given by

$$E_i^{total}(t) = y_{i,0}^t E_{i,0}^l(t) + \sum_{m \in \mathcal{V}_i} y_{i,m}^t E_{i,m}(t) + \sum_{k \in \mathcal{M}'} y_{i,k}^t E_{i,k}(t). \quad (22)$$

IV. PROBLEM FORMULATION AND SOLUTION

A. Problem Formulation

In this section, the problem is formulated to minimize the long-term overall system cost by concurrently optimizing computation offloading, service caching, cache updating, and resource allocation in MEC networks. The system cost is defined as $\sum_{i=1}^I (1 - \alpha) T_i^{total}(t) - \alpha H(t)$, where $H(t)$ is the cache hit ratio, and α is a parameter that balances the trade-off between the delay and cache hit ratio. The total system cost is the sum of the costs of all computation tasks over the long term. For each time slot, based on the computation offload decision set $\mathbf{X}(t)$, the service caching decision set $\mathbf{Y}(t)$, and the computation resource allocation decision set $\mathbf{Z}(t)$, the optimization problem P_1 is formulated as follows:

$$P_1 : \min_{\mathbf{X}(t), \mathbf{Y}(t), \mathbf{Z}(t)} \frac{1}{T} \sum_{t=1}^T \sum_{i=1}^I (1 - \alpha) T_i^{total}(t) - \alpha H(t) \quad (23a)$$

$$s.t. \ x_{i,m}^t \in [0, 1], z_{i,m}^t \in (0, 1], \forall i \in \mathcal{I}, \forall m \in \mathcal{M}, \forall t \in \mathcal{T} \quad (23a)$$

$$y_{i,m}^t, y_{i,m}^{p,t} \in \{0, 1\}, \forall i \in \mathcal{I}, \forall m \in \mathcal{M}, \forall t \in \mathcal{T} \quad (23b)$$

$$\Delta y_{i,m}^t \in \{-1, 0, 1\}, \forall i \in \mathcal{I}, \forall m \in \mathcal{M}, \forall t \in \mathcal{T} \quad (23c)$$

$$\sum_{m \in \mathcal{M}} x_{i,m}^t = 1, \forall i \in \mathcal{I}, \forall t \in \mathcal{T} \quad (23d)$$

$$\sum_{i \in \mathcal{I}} y_{i,m}^t \mu_i \leq D_m, \forall m \in \mathcal{M}, \forall t \in \mathcal{T} \quad (23e)$$

$$\sum_{m \in \mathcal{M}} y_{i,m}^t \geq 1, \forall i \in \mathcal{I}, \forall t \in \mathcal{T} \quad (23f)$$

$$\sum_{i \in \mathcal{I}} z_{i,m}^t \leq 1, \forall m \in \mathcal{M} \quad (23g)$$

$$-y_{i,m}^t \leq \Delta y_{i,m}^t \leq \min(y_{i,m}^{p,t}, 1 - y_{i,m}^t), \forall i \in \mathcal{I}, m \in \mathcal{M} \quad (23h)$$

$$E_i^{total}(t) \leq e_i^t, \forall i \in \mathcal{I}, m \in \mathcal{M}, t \in \mathcal{T}, \quad (23i)$$

where the constraints (23a) (23b) and (23c) specify the values of the optimization variables; constraint (23d) ensures that each computation task can be processed in its entirety and will not be processed repeatedly; constraint (23e) indicates that all the services of the UE's cache must not exceed the cache capacity; constraint (23f) specifies that the service data of any task should be cached on at least one edge node; constraint (23g) denotes that computation resource allocation for all tasks on each edge node cannot exceed the threshold; constraint (23h) represents the cache service update policy at t ; constraint (23i) refers to the energy constraint for the UE i at time slot t .

The problem we are addressing is a mixed-integer non-linear programming (MINLP) problem, a known NP-hard issue that poses challenges in finding the optimum solution quickly. In the dynamic communication state of the MEC scenario, numerical-based optimization algorithms require continuous iterations to obtain the best solution, resulting in significant time consumption. Moreover, the vast solution space increases the risk of getting stuck in a local optimum, affecting the overall effectiveness of the solution. DRL features are perfectly aligned with the rapidly changing communication settings. Therefore, we are embarking on this study to utilize the DRL approach to effectively address this optimization problem.

B. Proposed Solution

In order to tackle the optimization problem P_1 using the DRL technique, our first step is to convert the problem into an MDP, defining the states, actions, and rewards. Additionally, we will propose a soft policy function that incorporates entropy based on the standard policy function. By introducing entropy, we can measure the unpredictability of the output actions and prevent the algorithm from settling into a local optimum. Lastly, we will present an ISAC method to effectively address this issue.

1) *Problem Transformation*: The offloading computation process in MEC can be conceptualized as a colorblue Markov decision process (MDP). In this scenario, an agent interacts with its environment to identify optimal actions. This is denoted as $\{S, A, P, R\}$, with subsequent sections describing the state space, action space, and rewards.

2) *State Space*: The agent will output the corresponding action based on the observed state, and the state of this MDP can be expressed as

$$S(t) = \{S^{task}(t), S^{comp}(t), S^{pre}(t)\}, \quad (24)$$

where $S^{task}(t) = \{l_i^t, c_i^t, e_i^t\}$ contains all the task information at time slot t , including the task data size, the CPU cycles required to process each bit of data, the completion deadline, and the service cache policy; $S^{com}(t) = g_{i,m}^t$ is the communication state, containing the channel gain between each UE and the corresponding edge server; $S^{pre}(t) = \{H(t-1), \dots, H(t-L)\}$

contains the agent's previous L cache hit ratio, where the agent can learn from its previous cache hit ratio.

3) *Action Space*: We adopt actions to represent the computation offloading, service caching, and computation resource allocation decisions of the agent. The action space is represented as

$$a(t) = \{X(t), Y(t), Z(t)\}. \quad (25)$$

C. Rewards

In MDP, the agent dynamically learns the strategy of output actions to maximize reward. However, in this paper, we aim to minimize the objective function, without violating the cache capacity and energy consumption constraints, we set the reward function to the negative of the optimization objective function. When constraints are violated, a penalty is applied to encourage the agent to try to avoid outputting constraint-violating actions during learning, and the reward function is denoted as

$$r(t) = \begin{cases} -\sum_{i=1}^I (1 - \alpha) T_i^{total}(t) + \alpha H(t), & \text{(23e)–(23i) hold} \\ -\varphi, & \text{otherwise,} \end{cases} \quad (26)$$

where φ is a large real number.

D. Soft Policy Function

The traditional DRL algorithm may encounter local optima when solving problems in high-dimensional action spaces. Fortunately, soft actor-critic (SAC) offers certain advantages over traditional DRL. An entropy regularization term is introduced in the SAC algorithm's objective function, where the entropy reflects the randomness of the strategy's output actions. This enhances the algorithm's exploratory ability and reduces the risk of the strategy converging to local optima.

This introduced entropy term is represented as

$$\mathcal{H}(\pi(\cdot|s_t)) = \mathbb{E}_{a_t \in A} (-\log \pi(a_t|s_t)), \quad (27)$$

where $\pi(a_t|s_t)$ denotes the policy that output action a_t at the state s_t .

The objective of the SAC algorithm is to identify the policy that optimizes the sum of the reward and the entropy value. Consequently, the optimal policy is denoted as follows:

$$\pi^* = \arg \max_{\pi} \mathbb{E}_{\pi} \left[\sum_t \gamma^t [r(s_t, a_t) + \mu \mathcal{H}(\pi(\cdot|s_t))] \right], \quad (28)$$

where γ denotes the discount factor, and μ is a coefficient to weight the reward against entropy. A larger μ indicates a stronger exploration capability. However, the enhancement of the exploration ability of the algorithm will come at the cost of its exploitation ability.

In Deep Reinforcement Learning (DRL), the Q value quantifies the long-term reward associated with a state-action pair under a specified policy, whereas in the SAC method, the soft Q value is utilized to measure the sum of the long-term reward and entropy values [40], indicated as

$$Q(s_t, a_t) = r(s_t, a_t) + \mathbb{E}[\gamma Q(s_{t+1}, a_{t+1})]. \quad (29)$$

In the SAC algorithm, we define the soft V value to evaluate the value at state s_t , which is denoted as

$$V(s_t) = \mathbb{E}_{a_t \sim \pi} [Q_{\pi}(s_t, a_t) + \mu \mathcal{H}(\pi)]. \quad (30)$$

E. Improved Soft Actor-Critic for PI

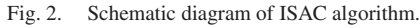
The soft SAC algorithm achieves excellent exploration capabilities by incorporating entropy into the objective. However, traditional SAC focuses only on the current states and actions when making decisions, neglecting historical states, which may affect the long-term cost. When the action space is large, the SAC algorithm often struggles to effectively train the neural networks. To address this issue, we incorporate Long Short-Term Memory (LSTM) to improve the average long-term system cost [41], where the LSTM is embedded within the neural networks. Furthermore, to reduce the total solution space [35], we design a multi-branch policy network to generate actions for each UE.

The ISAC algorithm consists of an actor module and a critic module. The actor module is responsible for training the policy, while the critic module includes four Q-networks, which guide the updates to the policy network. Based on the current observed state, the actor module outputs corresponding actions, including task offloading decisions, caching decisions, cache updating, cache pushing decisions, and computation resource allocation. The critic module outputs the Q-values and target Q-values, evaluating the actions performed by the actor module to ensure better actions and improved strategies. The training process and principles of these two modules are described below. The principle of the proposed Algorithm 1 is illustrated in Fig. 2.

1) *Critic Module*: In the critic module, there are also two Q networks denoted by $Q_{\theta_j}^j(s, a)$, $j = 1, 2$, where θ_j is the parameter of the j^{th} Q-network, each of the Q-network is equipped with a target Q-network with parameters $\bar{\theta}_j$, $j = 1, 2$. In addition, the smaller value of the outputs of the two network Q networks will be taken when updating the target Q-network, which can mitigate the overestimation of the value function. To enhance the utilization efficiency of the training samples while breaking the temporal order relationship between the training samples, an experience replay pool is introduced, denoted by $\mathcal{B}$. It can store newly generated experiences, each of which is a tuple that can be used for training, including state, action, reward, and state with the next moment, denoted as (s_i, a_i, r_i, s_{i+1}) . For each training session, a randomized mini-batch of samples containing $|\mathcal{B}|$ samples are taken as training samples.

For the Q-network, the training method is given as follows. To avoid bootstrapping and mitigate overestimation, the soft target value of the Q-network at the next time step s_{i+1} is determined by the minimum value of the target Q value, and then, the target Q value is given by

$$\begin{aligned} y(s_{i+1}, a_{i+1}) &= r + \gamma \mathbb{E}_{s_{i+1} \sim \rho} V_{\bar{\theta}}(s_{i+1}) \\ &= r + \gamma \left(\min_{j=1,2} Q_{\bar{\theta}_j}(s_{i+1}, a_{i+1}) \right. \\ &\quad \left. - \mu \log \pi_{\varphi_*}(a_{i+1}|s_{i+1}) \right), \end{aligned} \quad (31)$$



We define the loss function of Q network as the mean squared error (MSE) between the soft output value Q_θ and the soft target value $Q_{\bar{\theta}}$ to train Q_θ . As shown in Fig. 2, the $Q_{\theta_1}(s_i, a_i)$ is used to train Q_{θ_1} as ①, and $Q_{\theta_2}(s_i, a_i)$ is used to train Q_{θ_2} as ②. The loss function Q_θ is given by

The gradient calculation is expressed as

The parameters of the soft network Q are updated by minimizing the objective function with the gradient descent method,

where λ_q is the learning rate.

$$\overline{\theta}_j \leftarrow \tau\theta_j + (1 - \tau)\overline{\theta}_j, j = 1, 2. \quad (35)$$

Figure 1 illustrates the architecture of the proposed deep learning model. The model consists of three main layers: the Input Layer, the Hidden Layer, and the Output Layer.

- Input Layer:** This layer receives three types of input:
 - Task request information**
 - Communication condition**
 - Information about past cache hit ratio**
- Hidden Layer:** This layer contains two fully connected (FC) layers.
 - The first FC layer (red) receives inputs from the 'Task request information' and 'Communication condition' boxes, as well as from a sequence of LSTM units.
 - The LSTM units process a sequence of historical cache hit ratios: $H(t-1)$, $H(t-2)$, $H(t-3)$, ..., $H(t-L)$.
 - The output of the first FC layer is fed into the second FC layer (blue).
- Output Layer:** This layer produces three decisions based on the output from the second FC layer:
 - Computing offloading decision**
 - Service caching/updating/pushing decision**
 - Computing resource allocation decision**

Fig. 3. Schematic diagram of neural network for actor module.

The decision of each UE is made with a separate DNN network, and to stabilize and increase the exploration ability of the training process, random Gaussian noise (δ_t) is added to

each DNN. φ_i denotes the network parameter of the UE i ; On the other hand, the reparameterization for the action is used to ensure action differentiation, therefore the output of the action is represented as

$$a_i^t = f_{\varphi_i}(\delta_t; s_t). \quad (36)$$

By maximizing the desired soft Q-value with policy $\pi_{\varphi_i}(\cdot|s_t)$, we can optimize the actor-network parameters. Therefore, the loss function of the actor-network is defined as

$$L_{\pi}(\varphi_i) = \mathbb{E}_{s_t \sim \mathcal{D}, a_i^t \sim \pi_{\varphi_i}} [\mu \log(\pi_{\varphi_i}(a_i^t|s_t)) - \min_j Q_{\theta_j}(s_t, a_i^t)], j = 1, 2, \quad (37)$$

where $\mathcal{D}$ is the distribution of the replay buffer.

The strategy gradient is given by

$$\begin{aligned} \nabla_{\varphi_i} L(\varphi_i) &= \nabla_{\varphi_i} \mu \log(\pi_{\varphi_i}(a_i^t|s_t)) + \left(\nabla_{a_i^t} \mu \log(\pi_{\varphi_i}(a_i^t|s_t)) \right. \\ &\quad \left. - \nabla_{a_i^t} \min_j Q_{\theta_j}(s_t, a_i^t) \right) \nabla_{\varphi_i} f_{\varphi_i}(\delta_t; s_t), j = 1, 2. \end{aligned} \quad (38)$$

Minimizing the loss function by gradient descent $\mathcal{L}(\varphi_i)$ to update actor network parameters

$$\varphi_i \leftarrow \varphi_i - \lambda_{\pi} \nabla_{\varphi_i} \mathcal{L}(\varphi_i). \quad (39)$$

where λ_{π} is the learning rate.

3) *Entropy Coefficient Auto Adjustment*: The SAC algorithm provides a policy where the entropy factor μ should be sufficiently large at the early stage to enhance exploration ability and should decrease during the training stage to improve the utilization of high-quality samples. According to [42], the initial value of the entropy coefficient parameter can be automatically adjusted to an appropriate value using an auto-adjustment strategy. Based on our experiments, we set the initial entropy coefficient to 0.01.

According to [43], the loss function is given by

$$L(\mu) = \mathbb{E}_{a_t \sim \pi} [-\mu \log \pi(a_t|s_t) - \mu \mathcal{H}_0]. \quad (40)$$

Therefore, the entropy coefficient is updated as follows:

$$\nabla_{\mu} L(\mu) = -\log \pi(a_t|s_t) - H_0, \quad (41)$$

where H_0 is $-\dim(A)$ and A is the action space. Furthermore, the entropy can be updated as

$$\mu \leftarrow \mu - \eta \nabla_{\mu} L(\mu), \quad (42)$$

where η is learning rate.

F. Algorithm Design

We summarize the algorithm as follows. Firstly, we initialize the network parameters of the Q network with θ_1 and θ_2 , the actor network parameter φ_i , and update period UP , and then copy the parameter of Q-network to target Q-network with $\bar{\theta}_1$ and $\bar{\theta}_2$, and actor network with φ from line 1 and line 2. From line 3 to line 22, in each epoch, it will first observe the state s_i (at line 4). For each environment step, each user will generate the action $a_i^t = \phi_i(\delta_t; s_i)$ via actor-network with noise δ_i (at line 6), and then combine all the action $a_t = \{a_1^t, a_2^t, \dots, a_I^t\}$ (at line

Algorithm 1: The Improved Soft Actor Critic.

Algorithm 1: The Improved Soft Actor Critic

Input : The environment state s_i , Gaussian noise δ_i , min-batch sample set $\mathcal{B}$;

Output: Soft Q-network parameters $\theta_j, j = 1, 2$, and policy network π_{φ_i} parameters φ_i .

```

1 Initialization parameters:  $\theta_1, \theta_2, \varphi_i, \mathcal{B}$ , update period  $UP$ ;
2 + Copy the parameter  $\bar{\theta}_1 = \theta_1$ , and  $\bar{\theta}_2 = \theta_2$ ;
3 for  $e = 1$  to  $E$  do
4   Observe the state  $s_i$ ;
5   for  $t = 1$  to  $T$  do
6     Each user outputs  $a_i^t = f_{\varphi_i}(\delta_t; s_t)$  through the actor network;
7     Combine each action to a new action  $a_t = \{a_1^t, \dots, a_I^t\}$ ;
8     Execute action  $a_t$  by interacting with the environment, obtain state  $s_{t+1}$  and return the current reward  $r_t$ ;
9     Save  $(a_t, s_t, r_t, s_{t+1})$  into the experience replay pool;
10    if the size of the buffer  $\mathcal{B}$  is larger than 1000 then
11      for each update step do
12        Randomly sample a mini-batch data from the  $\mathcal{B}$ ;
13        Update parameters of Q-network  $\theta_1, \theta_2$  as Eq. (34);
14        if  $t \% UP == 0$  then
15          Update network parameters  $\phi_i$  of actor network by Eq. (39);
16          Update network parameters of target Q-network  $\bar{\theta}_j, j = 1, 2$  by Eq. (35);
17          Update the temperature factor  $\mu$  as  $\mu \leftarrow \mu - \eta \nabla_{\mu} L(\mu)$  according to Eq. (42).
18        end
19      end
20    end
21  end
22 end

```

7); execute the action a_t interacting with the environment (at line 8), and observe the next state s_{t+1} then obtain the reward r_t . Lastly, the transition (s_t, a_t, r_t, s_{t+1}) is put into the replay buffer (at line 9). From line 10 to line 18, when the buffer is full, it will first sample mini-batch data from $\mathcal{B}$, and then update the parameters of Q network θ_1, θ_2 at line 13. After every UP step, it will update the actor network ϕ_i as well as the parameter of target Q network $\bar{\theta}_j, j = 1, 2$, and then adjust the temperature factor μ to avoid the local optima from line 14 to line 17.

G. Analysis of Computation Complexity and Convergence Analysis

1) *Analysis of Computation Complexity*: The time complexity of the neural network is related to the actor module and the critic module. During the training phase, the algorithm involves

updating the parameters of both modules. We assume that there are I neural networks with the same structure in the actor module to train the policy for all users. On the other hand, we also assume that the critic network consists of four Q neural networks with the same structure. Therefore, the complexity of the algorithm is given by

$$I * \sum_{j=0}^{J-1} n_j^A n_{j+1}^A + 4 * \sum_{k=0}^{K-1} n_k^C n_{k+1}^C$$

$$= O \left(I \sum_{j=0}^{J-1} n_j^A n_{j+1}^A + \sum_{k=0}^{K-1} n_k^C n_{k+1}^C \right), \quad (43)$$

where J represents the number of fully connected layers with the same structure in the actor module, and K is the number of fully connected layers with the same structure in the critic module. n_j^A and n_k^C are the number of neurons in the j^{th} layer of the actor module and critic module, respectively. On the other hand, $j = 0$ and $k = 0$ indicate the input layer.

Since the LSTM is added to the network, the number of neurons in the first hidden layer is $n_1^A = Ln^l + n_1$, L is the number of LSTM module units, n^l is the number of neurons in the LSTM unit, n_1 is the number of neurons besides the LSTM in the first hidden layer. During the test phase, it only needs to use the trained actor-network to output the action decision. Therefore, the computational complexity during the testing phase is $O(I \sum_{j=0}^{J-1} n_j^A n_{j+1}^A)$.

2) *Convergence Analysis*: The algorithm belongs to the maximum entropy algorithm, the algorithm alternates soft policy evaluation and soft policy improvement, that is, it will alternate between updating the function approximators with the sampled data and collecting the experience with the current policy. The algorithm is convergent as in [43].

V. PERFORMANCE EVALUATION

In this section, we evaluate the performance of the proposed scheme through simulation experiments. First, we introduce the experimental environment and parameter settings. Next, to select more suitable hyperparameters, we perform comparative studies on the hyperparameters. Finally, we compare the proposed algorithm with other benchmark schemes to validate the effectiveness of the proposed method in different scenarios using various metrics.

A. Experimental Setting

We create a simulation environment with MEC systems using Python 3.6 and employ PyTorch 1.13.1 to implement the DRL algorithm. In the experiments, we set the total number of time slots to 200 and consider a MEC network consisting of 3 edge servers and 15 UEs. In addition, the UEs and BS are randomly placed in the simulation area, and the communication radius of the BS is 300 m. Moreover, the UEs can communicate with the BS via a wireless link or forwarding through a fiber link. Additionally, users generate tasks during each time slot, which require the current nodes to cache the corresponding type of service for local processing. We assume that the data size

TABLE III
SIMULATION SETTINGS

Parameters	Values
Time slot T	200
Episode E	500
Number of edge server(ES) M	[2,12]
Number of UE I	[5,25]
Number of service type S	8
Caching capacity of UEs D_0	[1,3] GB
Each server caching requirement	30 GB
Computing capacity of UEs F_i^t	1 GHz
Computing capacity of ESs F_m^t	[3,7] GHz
Data size of task l_i^t	[4,8]MB
CPU cycle of processing each bit data c_i^t	800 cycles/bit
Bandwidth between ESs and UEs W_{im}^t	[25,300] MHz
κ	10^{-27}
Noise power	$2 * 10^{-13} W$
Discount factor γ	0.99
Weighted factor α	0.3
Noise power σ^2	200dBm
Transmit power p_i	100mW
Learning rate λ_q	Cosine annealing $6 \times 10^{-5} \rightarrow 1 \times 10^{-6}$
Learning rate λ_π	Cosine annealing $4 \times 10^{-5} \rightarrow 1 \times 10^{-6}$
Learning rate η	Cosine annealing $4 \times 10^{-5} \rightarrow 1 \times 10^{-6}$
Initial value of entropy coefficient μ	0.01

generated by UE i is l_i^t (in bits), limited to the range of 4 to 8 MB. The task arrival process follows a Poisson distribution with rate λ_i . On the other hand, the cached service types in this experiment are set to 8, and each service occupies a certain cache space, which includes the applications, program codes, databases necessary for computation, and so on. The size of the cached service is 15 MB. The maximum computation power of the UEs is set to 1 GHz, while the edge server computation power is randomly distributed between 3–7 GHz. We update the model once per time step and stop training and testing after 500 rounds of training. Other detailed parameters are shown in Table III.

B. Convergence Performance

Firstly, we examined the convergence performance of the algorithms under various network hyperparameter settings, as shown in Fig. 4, where the x-axis represents the number of training rounds (each round contains 200 time steps), and the y-axis indicates the sum of rewards for all time steps in the round. The principle of DRL is that the agent updates the network parameters based on the reward values returned from interactions with the environment to maximize rewards. Thus, the reward value is the most critical indicator of the DRL algorithm's convergence.

As shown in Fig. 4(a), the convergence performance of different algorithms is compared. We observe that our proposed method outperforms other methods in terms of the reward value after the algorithm converges. The performance of ISAC and

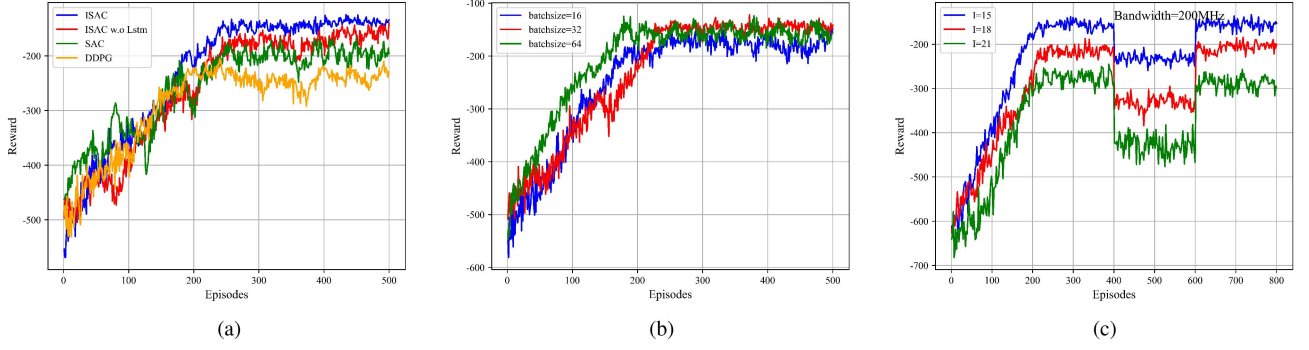


Fig. 4. Convergence for different parameters. (a) Average reward versus different methods (b) Average reward versus different batch sizes (c) Adaptability versus different UEs and bandwidths.

ISAC w.o LSTM are similar to our proposed method, since these two methods are also based on the SAC method and they also consider incorporating the entropy as part of the objective. However, the performance of DDPG is the worst one. The reason may be that our proposed method considers the multi-branch structure which will reduce the action space and improve the training efficiency, in addition the LSTM module in our proposed method can make use of the history caching information to improve the reward.

As shown in Fig. 4(b), we evaluate the proposed algorithm with different batch sizes, which refer to the sample numbers from the experience replay pool during the training period. With the increase in batch size from 16 to 32, the number of episodes needed for convergence increases, and the reward value after the algorithm converges increases. On the other hand, when the batch size increases from 32 to 64, the number of episodes needed for convergence almost remains the same, but the reward value after convergence increases significantly, and the time required for training per round increases due to the larger batch size. To summarize, we set the batch size to 64 in subsequent simulations.

As shown in Fig. 4(c), we adopt different numbers of UEs and various bandwidths to illustrate the adaptability of the algorithm. The number of UEs is set to 15, 18, and 21 in these settings. Before 400 rounds of simulations, the bandwidth for communication is set to 300 MHz. We observe that the reward converges to a stable value with different numbers of UEs. From the 400th to the 600th round, the bandwidth is reduced to 200 MHz, and the reward still converges to a stable value. From rounds 600 to 800, when the bandwidth is changed back to 300 MHz, the reward continues to converge to a stable value. From different UE numbers and bandwidths, we conclude that our proposed algorithm demonstrates a significant degree of adaptability.

C. Performance Comparison

In this section, we compare the performance of the ISAC algorithm with other schemes in MEC scenarios. The benchmark algorithms involved in the comparison include:

- 1) Improved soft actor-critic without LSTM (ISAC w.o LSTM): Based on the proposed ISAC algorithm, no LSTM

module is utilized in the actor-network. Other parts are consistent with the algorithm presented in this paper.

- 2) Soft actor-critic (SAC) [44]: The basic SAC algorithm, which also introduces entropy in the objective function to enhance the exploration capability. The actor module employs a DNN network to output the actions of all the UE, and additionally, the critic module includes four Q-networks without the LSTM module and multi-branch module.
- 3) Deep deterministic policy gradient (DDPG) [45]: Deterministic policy DRL algorithm that can be used to solve continuous action space with an objective function of long-term reward expectation excluding the entropy term. The algorithm consists of two actor networks and two critic networks, incorporating a target to stabilize the training process and mitigate bootstrapping issues.
- 4) ALL offloading (AllOff): In this scheme, computation tasks are not computed locally; each task is randomly uploaded to the edge servers to process, and services are randomly cached.

In this section, we evaluate two performance metrics: system cost and average latency. As seen from (23), the system cost comprises the delay and the cache hit ratio, reflecting the effect of the algorithm in optimizing the latency, as well as the accuracy of the algorithm for cache prediction. The increase in cache hit ratio does not conflict with the reduction of latency. In addition, the objective function can guide the updating of the policy during the training process to explore the policy with lower latency more efficiently.

D. System Cost

In Fig. 5, the system costs of the compared methods are evaluated under scenarios with different bandwidths, task data volumes, UE service cache capacities, task arrival rate. Fig. 5(a) shows the system costs under varying uplink bandwidths (25–300 MHz). As bandwidth increases, transmission rates improve, reducing system costs across all methods. ISAC consistently outperforms others by leveraging LSTM to predict caching based on past cache hits and coordinating task offloading among edge servers. In contrast, AllOff is more sensitive to bandwidth

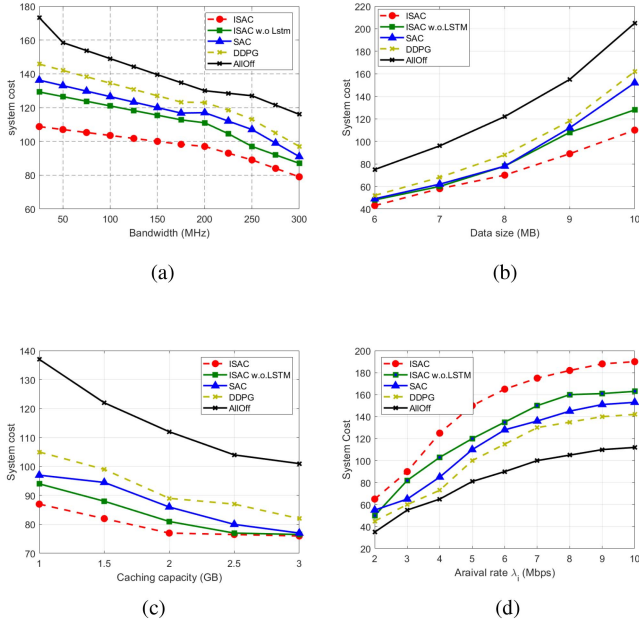


Fig. 5. Comparison of system cost performance. (a) System cost versus bandwidth (b) System cost versus data size (c) System cost versus UE caching capacity (d) System cost versus arrival rate.

changes and exhibits the largest cost variation due to its random offloading and caching decisions.

In Fig. 5(b), we compare system costs across methods with task data volumes from 4 MB to 8 MB. System costs rise with data size due to increased computation, communication latency, and energy consumption, which can lead to processing failures. The Alloff method performs worst due to its random offloading and caching. ISAC and SAC outperform DDPG, with ISAC achieving the lowest cost thanks to its LSTM module that leverages past cache hit ratios for better offloading decisions. The performance of ISAC w.o LSTM and SAC is similar, as the multi-branch actor in ISAC w.o LSTM offers limited improvement.

Fig. 5(c) compares system costs under varying UE cache capacities. As cache capacity grows, system costs decrease due to reduced transmission delay and higher cache hit rates. SAC, DDPG, and Alloff show lower costs than ISAC and ISAC w.o LSTM with increasing cache size, likely because they lack caching update and pushing policies. However, ISAC consistently outperforms others, especially with limited cache, by proactively pushing caches and using LSTM to predict caching, enabling better offloading decisions and lower system costs.

In Fig. 5(d), we validate the impact of the arrival rate on the system cost. The arrival rate varies from 2 Mbps to 10 Mbps while keeping other parameters unchanged. We observe that the total system cost rises with the increase of the task arrival rate due to the growing system load. Our proposed method achieves the lowest system cost compared to other methods. This is because, as the task arrival rate increases, more tasks are offloaded to edge servers, and some tasks are further transferred to cooperative edge servers following the caching update policy. Among these schemes, SAC, DDPG, and Alloff methods achieve higher

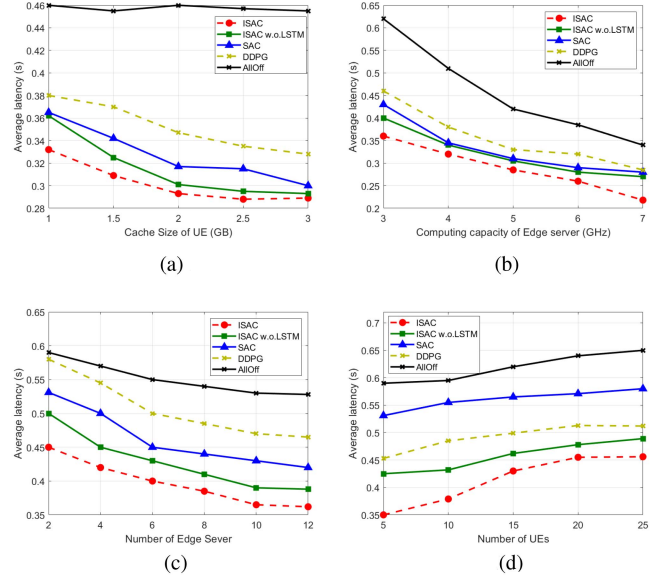


Fig. 6. Comparison of the average latency performance with the benchmark algorithm. (a) Average latency versus UE caching capacity (b) Average latency versus ES computing capacity (c) Average latency versus number of edge server (d) Average latency versus number of UEs.

system costs compared to ISAC and ISAC without LSTM. This may be because these methods lack caching update policies and cooperation among edge nodes.

E. Average Delay

In Fig. 6, we compare average latency under varying conditions. Fig. 6(a) shows latency versus UE cache size (1–3 GB). Local cache size has little effect on Alloff, since all tasks are processed at the edge server. DDPG and SAC perform worse than ISAC and ISAC w.o LSTM due to lack of caching policy. ISAC outperforms ISAC w.o LSTM by using multi-branch and LSTM modules to predict better caching policies, improving offloading and reducing latency.

Fig. 6(b) compares latency with varying edge server computation power (3–7 GHz). Latency decreases for all methods as power increases. ISAC w.o LSTM and DDPG show similar performance, as large edge cache reduces the impact of multi-branch structure. DDPG and Alloff have higher latency, with Alloff worst due to lack of cooperation and caching policy. ISAC achieves the lowest latency by leveraging LSTM to better predict cache placement on UEs and edge nodes.

In Fig. 6(c), we examine how varying the number of edge servers (2 to 12) affects average processing latency, with other parameters fixed. Latency generally decreases and then stabilizes as more edge servers provide greater processing opportunities. Our method consistently achieves the lowest latency due to enhanced cooperation among edge servers. Fig. 6(d) shows that with 4 edge servers and UE numbers from 5 to 25, latency rises with more UEs before stabilizing, as increased task volume leads to higher processing time. Our method outperforms others by processing some tasks locally and offloading others cooperatively, while leveraging caching updates to improve task

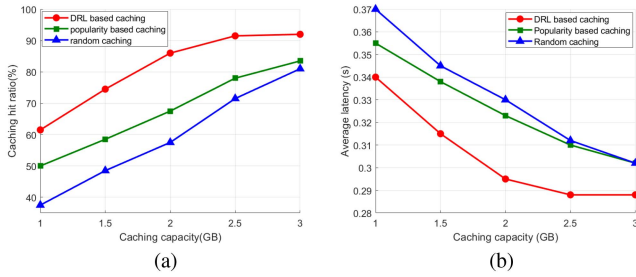


Fig. 7. Comparison of cache hit ratio and latency of each method. (a) Caching hit ratio versus different caching capacity (b) Average latency versus different caching capacity.

offloading and cache hits. Other methods often overlook edge server cooperation or caching capacity.

F. Impact of Cache Methods on Cache Hit Ratio and Average Delay

In this section, an experimental comparison is conducted to evaluate the performance of different service caching methods. Note that the task offloading and computation resource allocation are also determined by the ISAC method; however, the caching decision is determined by different algorithms. In this section, we compare the effectiveness of our DRL-based caching approach with LRFU [46], which is a popularity-based method, and a random caching approach in terms of average processing delay and cache hit ratio under different UE cache capacity settings as shown in Fig. 7.

- 1) Least recently/frequently used (LRFU): A popularity-based caching approach that introduces a time decay factor, combining the request frequency and request time of different services to synthesize the caching and clearing of services.
- 2) random caching: In this method, the UE automatically updates the cached services at each time slot, randomly caching the services. When the cache capacity is insufficient, the services are randomly deleted to store new services.

The cache hit ratio for each method with different UE cache capacity settings is shown in Fig. 7(a). The larger the cache capacity, the more services can be cached by the UE, making it easier to cache the services required by the computation tasks. Random caching, due to its completely randomized nature, experiences a significantly low cache hit ratio when the cache capacity is small. Compared with LRFU and random caching, DRL-based caching maintains a high cache hit ratio under different conditions, especially when the cache is very limited. On the other hand, our proposed DRL-based caching policy achieves a high cache hit ratio because our proposed method uses the LSTM module to train and predict the cache hit ratio. Additionally, our method dynamically adjusts the caching pushing variable according to the dynamic environment status.

As shown in Fig. 7(b), we compare the impact of various caching methods on average delay with different caching methods. The average latency for all methods decreases as the caching capacity increases. Our proposed method outperforms other

methods in terms of average latency, while random caching obtains the worst performance with average latency. This is because random caching cannot ensure the task is efficiently processed on the edge nodes, meaning the task cannot be computed when the service is not cached on the edge nodes. The LRFU method achieves medium performance since it caches the service according to its popularity, which improves the cache hit ratio to a certain extent and reduces processing latency. Our proposed DRL-based caching achieves the best performance because it considers both the caching update and caching pushing decisions. Moreover, the LSTM module and the multi-branch module enable the system to learn from past cache policies to improve the offloading decision.

VI. CONCLUSION

In this paper, we addressed the problem of collaborative task offloading, service caching, and resource allocation in MEC to minimize task latency and enhance the service cache hit ratio. We began by defining the latency and energy consumption associated with both local computation and offloading of computation tasks. We then introduced the system cost, which comprises the total long-term task processing latency and cache hit ratio, and formulated the problem as a mixed-integer nonlinear programming (MINLP) challenge.

To tackle this problem, we transformed the optimization challenge into an MDP aimed at minimizing system cost. We proposed an improved soft actor-critic (ISAC) method to make decisions regarding computation offloading, service caching, and resource allocation. The ISAC approach includes an entropy term to enhance the exploration capability of the algorithm, preventing it from settling into local optima. Additionally, we employed an adaptive updating entropy coefficient to strike a balance between exploration and exploitation, facilitating the convergence of the algorithm. The Actor module uses a multi-branch structure to output different strategies for each actor, reducing the dimensionality of the problem space, and incorporates LSTM to leverage cache hit ratio experience.

Finally, we validated the performance of the ISAC algorithm by comparing its system cost and average latency against benchmark algorithms. We also compared the cache ratio and average latency of our DRL-based caching method with other caching strategies, demonstrating the effectiveness of our approach. In the future, we will combine the multi-agent and cloud service to extend this research.

REFERENCES

- [1] G. S. Park, R. H. Kim, and H. Song, "Collaborative virtual 3D object modeling for mobile augmented reality streaming services over 5D networks," *IEEE Trans. Mobile Comput.*, vol. 22, no. 7, pp. 3855–3869, Jul. 2023.
- [2] Z. Tang, J. He, K. Yang, and H. Chen, "Matching 5D connected vehicles to sensed vehicles for safe cooperative autonomous driving," *IEEE Netw.*, vol. 38, no. 3, pp. 227–235, May 2024.
- [3] B. Girish et al., "IoT based security system for smart vehicle," *Int. Res. J. Eng. Technol.*, vol. 5, no. 5, pp. 2869–2874, 2018.
- [4] B. Liu, W. Ni, R. P. Liu, Q. Zhu, Y. J. Guo, and H. Zhu, "Novel integrated framework of unmanned aerial vehicle and road traffic for energy-efficient delay-sensitive delivery," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 8, pp. 10692–10707, Aug. 2022.

- [5] Z. Jiang, K. Li, Y. Wang, M. Liu, and H. Li, "A task allocation schema based on response time optimization in cloud computing," *Cluster Comput.*, vol. 27, no. 3, pp. 3893–3910, 2024.
- [6] B. P. Rimal, D. P. Van, and M. Maier, "Mobile-edge computing vs. centralized cloud computing in fiber-wireless access networks," in *Proc. IEEE Conf. Comput. Commun. Workshops*, 2016, pp. 991–996.
- [7] B. P. Rimal, D. P. Van, and M. Maier, "Mobile-edge computing versus centralized cloud computing over a converged FiWi access network," *IEEE Trans. Netw. Service Manag.*, vol. 14, no. 3, pp. 498–513, Sep. 2017.
- [8] W. Wen, Y. Cui, T. Q. S. Quek, F. Zheng, and S. Jin, "Joint optimal software caching, computation offloading and communications resource allocation for mobile edge computing," *IEEE Trans. Veh. Technol.*, vol. 69, no. 7, pp. 7879–7894, Jul. 2020.
- [9] L. Wang and G. Zhang, "Joint service caching, resource allocation and computation offloading in three-tier cooperative mobile edge computing system," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 6, pp. 3343–3353, Nov./Dec. 2023.
- [10] G. Zhang, S. Zhang, W. Zhang, Z. Shen, and L. Wang, "Joint service caching, computation offloading and resource allocation in mobile edge computing systems," *IEEE Trans. Wireless Commun.*, vol. 20, no. 8, pp. 5288–5300, Aug. 2021.
- [11] W. Liu, H. Zhang, H. Ding, and D. Yuan, "Delay and energy minimization for adaptive video streaming: A joint edge caching, computing and power allocation approach," *IEEE Trans. Veh. Technol.*, vol. 71, no. 9, pp. 9602–9612, Sep. 2022.
- [12] X. Gao, Y. Sun, H. Chen, X. Xu, and S. Cui, "Joint computing, pushing, and caching optimization for mobile-edge computing networks via soft actor-critic learning," *IEEE Internet Things J.*, vol. 11, no. 6, pp. 9269–9281, Mar. 2024.
- [13] X. Gao, Y. Sun, H. Chen, X. Xu, and S. Cui, "Soft actor-critic learning-based joint computing, pushing, and caching framework in MEC networks," in *Proc. IEEE Glob. Commun. Conf.*, 2023, pp. 1675–1680.
- [14] Z. Xue, C. Liu, C. Liao, G. Han, and Z. Sheng, "Joint service caching and computation offloading scheme based on deep reinforcement learning in vehicular edge computing systems," *IEEE Trans. Veh. Technol.*, vol. 72, no. 5, pp. 6709–6722, May 2023.
- [15] J. Du et al., "MADDPG-based joint service placement and task offloading in MEC empowered air-ground integrated networks," *IEEE Internet Things J.*, vol. 11, no. 6, pp. 10600–10615, Mar. 2024.
- [16] A. R. Heidarpour, M. R. Heidarpour, M. Ardakani, C. Tellambura, and M. Uysal, "Soft actor-critic-based computation offloading in multiuser MEC-enabled IoT - A lifetime maximization perspective," *IEEE Internet Things J.*, vol. 10, no. 20, pp. 17571–17584, Oct. 2023.
- [17] F. Zhang, G. Han, L. Liu, Y. Zhang, Y. Peng, and C. Li, "Cooperative partial task offloading and resource allocation for IIoT based on decentralized multiagent deep reinforcement learning," *IEEE Internet Things J.*, vol. 11, no. 3, pp. 5526–5544, Feb. 2024.
- [18] C. Yang, Q. Chen, Z. Zhu, Z. Huang, S. Lan, and L. Zhu, "Evolutionary multitasking for costly task offloading in mobile-edge computing networks," *IEEE Trans. Evol. Comput.*, vol. 28, no. 2, pp. 338–352, Apr. 2024.
- [19] Y. Zhang, M. A. Kishk, and M. Alouini, "Computation offloading and service caching in heterogeneous MEC wireless networks," *IEEE Trans. Mobile Comput.*, vol. 22, no. 6, pp. 3241–3256, Jun. 2023.
- [20] M. Song et al., "Joint user-side recommendation and D2D-assisted offloading for cache-enabled cellular networks with mobility consideration," *IEEE Trans. Wireless Commun.*, vol. 22, no. 11, pp. 8080–8095, Nov. 2023.
- [21] L. Gao, X. Chen, B. Yin, L. Cui, and H. Xing, "Game theory based task offloading, content caching and resource pricing under edge-cloud collaboration in 6D network," in *Proc. IEEE Wireless Commun. Netw. Conf.*, 2023, pp. 1–6.
- [22] J. Zhang, Y. Shen, Y. Wang, X. Zhang, and J. Wang, "Dual-timescale resource allocation for collaborative service caching and computation offloading in IoT systems," *IEEE Trans. Ind. Informat.*, vol. 19, no. 2, pp. 1735–1746, Feb. 2023.
- [23] J. Feng and J. Gong, "AoI-aware optimization of service caching-assisted offloading and resource allocation in edge cellular networks," *Sensors*, vol. 23, no. 6, 2023, Art. no. 3306.
- [24] W. Fan et al., "Collaborative service placement, task scheduling, and resource allocation for task offloading with edge-cloud cooperation," *IEEE Trans. Mobile Comput.*, vol. 23, no. 1, pp. 238–256, Jan. 2024.
- [25] J. Zhao, L. He, D. Zhang, and X. Gao, "A TP-DDPG algorithm based on cache assistance for task offloading in urban rail transit," *IEEE Trans. Veh. Technol.*, vol. 72, no. 8, pp. 10671–10681, Aug. 2023.
- [26] H. Zhou, Z. Zhang, Y. Wu, M. Dong, and V. C. M. Leung, "Energy efficient joint computation offloading and service caching for mobile edge computing: A deep reinforcement learning approach," *IEEE Trans. Green Commun. Netw.*, vol. 7, no. 2, pp. 950–961, Jun. 2023.
- [27] L. Wang, H. Liang, and D. Zhao, "Deep-reinforcement-learning-based computation offloading and power allocation within dynamic Platoon network," *IEEE Internet Things J.*, vol. 11, no. 6, pp. 10500–10512, Mar. 2024.
- [28] Q. Yang, S. Chu, C. Hu, L. Kong, and J. Pan, "A task offloading method based on user satisfaction in C-RAN with mobile edge computing," *IEEE Trans. Mobile Comput.*, vol. 23, no. 4, pp. 3452–3465, Apr. 2024.
- [29] M. Teng, X. Li, and K. Zhu, "Joint optimization of sequential task offloading and service deployment in end-edge-cloud system for energy efficiency," *IEEE Trans. Sustain. Comput.*, vol. 9, no. 3, pp. 283–298, May/Jun. 2024.
- [30] L. Liu, X. Yuan, N. Zhang, D. Chen, K. Yu, and A. Taherkordi, "Joint computation offloading and data caching in multi-access edge computing enabled Internet of Vehicles," *IEEE Trans. Veh. Technol.*, vol. 72, no. 11, pp. 14939–14954, Nov. 2023.
- [31] H. Zhou, T. Wu, X. Chen, S. He, D. Guo, and J. Wu, "Reverse auction-based computation offloading and resource allocation in mobile cloud-edge computing," *IEEE Trans. Mobile Comput.*, vol. 22, no. 10, pp. 6144–6159, Oct. 2023.
- [32] H. Zhou, Z. Wang, H. Zheng, S. He, and M. Dong, "Cost minimization-oriented computation offloading and service caching in mobile cloud-edge computing: An A3C-based approach," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 3, pp. 1326–1338, May/Jun. 2023.
- [33] Y. Wang, W. Fang, Y. Ding, and N. Xiong, "Computation offloading optimization for UAV-assisted mobile edge computing: A deep deterministic policy gradient approach," *Wireless Netw.*, vol. 27, no. 4, pp. 2991–3006, 2021.
- [34] B. Hazarika, K. Singh, S. Biswas, S. Mumtaz, and C. Li, "SAC-based resource allocation for computation offloading in IoV networks," in *Proc. 2022 Joint Eur. Conf. Netw. Commun. 6G Summit*, 2022, pp. 314–319.
- [35] X. Zhou, S. Ge, P. Liu, and T. Qiu, "DAG-based dependent tasks offloading in MEC-enabled IoT with soft cooperation," *IEEE Trans. Mobile Comput.*, vol. 23, no. 6, pp. 6908–6920, Jun. 2024.
- [36] A. Singh and K. Chatterjee, "Securing smart healthcare system with edge computing," *Comput. Secur.*, vol. 108, 2021, Art. no. 102353.
- [37] X. Xu, Z. Liu, M. Bilal, S. Vimal, and H. Song, "Computation offloading and service caching for intelligent transportation systems with digital twin," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 11, pp. 20757–20772, Nov. 2022.
- [38] A. Younis, S. Maheshwari, and D. Pompili, "Energy-latency computation offloading and approximate computing in mobile-edge computing networks," *IEEE Trans. Netw. Service Manag.*, vol. 21, no. 3, pp. 3401–3415, Jun. 2024.
- [39] H. Zhou, K. Jiang, S. He, G. Min, and J. Wu, "Distributed deep multi-agent reinforcement learning for cooperative edge caching in Internet-of-vehicles," *IEEE Trans. Wireless Commun.*, vol. 22, no. 12, pp. 9595–9609, Dec. 2023.
- [40] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proc. 35th Int. Conf. Mach. Learn.*, 2018, pp. 1856–1865.
- [41] M. Tang and V. W. S. Wong, "Deep reinforcement learning for task offloading in mobile edge computing systems," *IEEE Trans. Mobile Comput.*, vol. 21, no. 6, pp. 1985–1997, Jun. 2022.
- [42] L. Zhang, M. Liu, X. Zhang, L. Ling, and J. Hu, "Multivision tasks offloading of IoT perception in intelligent manufacturing," *IEEE Trans. Ind. Informat.*, vol. 21, no. 7, pp. 5480–5491, Jul. 2025.
- [43] T. Haarnoja et al., "Soft actor-critic algorithms and applications," 2018, *arXiv:1812.05905*.
- [44] Y. Zheng, H. Zhou, R. Chen, K. Jiang, and Y. Cao, "SAC-based computation offloading and resource allocation in vehicular edge computing," in *Proc. 2022 IEEE Conf. Comput. Commun. Workshops*, 2022, pp. 1–6.
- [45] H. Li, C. Chen, H. Shan, P. Li, Y. C. Chang, and H. Song, "Deep deterministic policy gradient-based algorithm for computation offloading in IoV," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 3, pp. 2522–2533, Mar. 2024.
- [46] S. Krishnendu, B. N. Bharath, N. Garg, V. Bhatia, and T. Ratnarajah, "Learning to cache: Federated caching in a cellular network with correlated demands," *IEEE Trans. Commun.*, vol. 70, no. 3, pp. 1653–1665, Mar. 2022.



Haizhou Bao received the B.S. degree in electronic information engineering from Huanggang Normal University, Huanggang, China, in 2012, and the Ph.D. degree in computer architecture from Wuhan University, Wuhan, China, in 2021. From 2019 to 2020, he was a Visiting Scholar with the University of Victoria, Victoria, BC, Canada. He was a Lecturer with the School of Computer Science and Technology, Wuhan University of Science and Technology, Wuhan, China. He is currently a Lecturer with the School of Artificial Intelligence and Computer Science, Hubei Normal University, Hubei, China. His research interests mainly include vehicular networks and mobile edge computing.



Yuxuan Wang received the B.S. degree in computer science and technology from Hankou University, Wuhan, China, in 2022. He is currently working toward the master's degree in computer science from Wuhan University of Science and Technology, Wuhan, China. His research interests mainly include mobile edge computing and reinforcement learning.



Yiming Huo (Senior Member, IEEE) received the B.Eng. degree in information engineering from Southeast University, Nanjing, China, the M.Sc. degree in system-on-chip (SoC) from Lund University, Lund, Sweden, and the Ph.D. degree in electrical engineering from the University of Victoria, Victoria, BC, Canada. He is currently a Research Associate with the University of Victoria. He has also worked with Ericsson, ST-Ericsson, Chinese Academy of Sciences, STMicroelectronics, and Apple. He holds several granted patents. His research interests include next-generation wireless systems and networks, Internet of Things, space technology, astronomy, and artificial intelligence. Dr. Huo was the recipient of multiple IEEE conference paper awards and the honor of the "Most Popular Article of IEEE Access in 2017", the ISSCC-STGA Award from the IEEE Solid-State Circuits Society. He is Co-Chair of the IEEE Future Networks Massive MIMO Working Group, and an IEEE Standards Association (SA) Officer. He was with Program Committee / TPC of the IEEE ICUBW / IEEE VTC / IEEE ICC, Session Chair of the IEEE 5G World Forum 2018, Publication Chair of the IEEE PACRIM 2019, and TPC Co-Chair / Publicity Co-Chair / Session Chair of IEEE Future Networks 1st / 2nd Massive MIMO Workshop 2021 / 2022, Technical Reviewer for multiple premier IEEE conferences, and transactions / journals. He is also an Associate Editor for IEEE ACCESS, and Guest Editors of *Signals and Symmetry*.



Peng Li (Member, IEEE) received the M.S. degree from the Huazhong University of Science and Technology, Wuhan, China, in 2006, and the Ph.D. degree from Wuhan University, Wuhan, in 2015. He is currently a Professor with Wuhan University of Science and Technology, Wuhan. From 2016 to 2017, he was a Visiting Scholar with the University of Calgary, Calgary, AB, Canada. His research interests include vehicular networks, mobile computing, crowd sensing, and Big Data.



Lei Nie (Member, IEEE) received the B.S. degree in computer science and technology from the Wuhan University of Science and Technology, Wuhan, China, in 2011, and the Ph.D. degree in computer architecture from Wuhan University, Wuhan, China, in 2017. He is currently a Lecturer with the School of Computer Science and Technology, Wuhan University of Science and Technology. His research interests mainly include vehicular networks and intelligent transportation.



Libing Wu (Member, IEEE) received the Ph.D. degree from Wuhan University, Wuhan, China, in 2006. He is currently a Professor with the School of Cyber Science and Engineering, Wuhan University. He was a Visiting Scholar with the Advanced Networking Lab, University of Kentucky, Lexington, KY, USA, in 2011. He was a Senior Visiting Fellow with the State University of New York, Buffalo, NY, USA, in 2017. His research interests include network security, Internet of Things, machine learning, and data security.



Guorong Yu received the M.S. degree in surveying and mapping engineering from the China University of Geoscience, Wuhan, China, in 2015, and the Ph.D. degree in photogrammetry and remote sensing from Wuhan University, Wuhan, China, in 2021. He is currently a Lecturer with the School of Computer Science and Technology, Wuhan University of Science and Technology, Wuhan, China. His research interests include photogrammetry and remote sensing, image registration, and 3D reconstruction.